

REMORA: Real-time Expressive Motion to Output Routing Audio

A Martial Arts Staff Instrument Driven by Motion

Timothée Dravet

Polytech Marseille, Aix-Marseille Université
Marseille, France

timothee.dravet@etu.univ-amu.fr

Daniel Overholt

CREATE, Aalborg University
Aalborg, Denmark

dano@create.aau.dk

ABSTRACT

REMORA turns a bō, the long staff used in martial arts, into a musical instrument: spins, tilts, and strikes become sound in real time. The instrument is built in two parts: a small wireless sensor unit rides on the staff and continuously reads its physical motion, while a companion audio plugin turns that motion into pitch, texture, and timbre across three very different synthesis voices, from a warm additive choir to a breath-driven vocal tract. Rather than one fixed gesture-to-sound behaviour, every mapping from physical motion to sound parameters is a small patch of connected building blocks - motion sources, shaping math, and synthesis sinks - wired on a visual canvas inside the plugin, in the spirit of a modular synthesiser's patch cables. The same small vocabulary of building blocks stretches from a beginner's direct tilt-controlled pitch to a compound, deliberately one-to-many mapping whose timbre quietly re-randomises itself with every completed spin, so that no two passes through the same gesture sound quite identical. Nine such presets currently ship, pared down over the course of development from a larger, more repetitive set of fifteen.

1. INTRODUCTION

REMORA grew out of the first author's parallel practice in martial arts and music. The bō itself is a form largely absent from new interfaces for musical expression: a full-body, two-handed, swung staff with its own weight, reach, and rotational inertia, quite unlike the small handheld or body-worn sensors most inertial digital instruments use. Where a martial artist reads speed, force, and reach off the staff by feel, REMORA reads the same qualities electronically and turns them into sound: a spin becomes a filter sweep, a slow controlled turn sounds nothing like a fast, loose one. The instrument's vocabulary is, in that sense, borrowed directly from the performer's own body of practice rather than invented for the occasion. Crucially, that vocabulary is not baked into compiled code either: every motion-to-sound relationship lives inside the plugin as a small, rewirable patch, so extending or retuning it is an editing operation the performer or instrument-builder can perform on the same canvas used to play the instrument, rather than a recompile.

Rod-, staff-, and baton-shaped controllers already form a small, recurring family in this space: the T-Stick's touch-sensitive shaft [1], the Digital Baton's conducting gesture [2], and Nam's

swung Musical Poi [3] among them. What REMORA adds to that family is less a new sensor than a new relationship: a martial artist's years of staff-handling skill, carried straight onto a stage as a musical vocabulary.



(a) Parametric design (c) Hardware wiring (b) Printed and mounted

Figure 1: The staff-mounted sensor unit, held on with zip ties so it can be repositioned or removed without marking the staff: (a) the parametric OpenSCAD design; (b) the hardware wiring; (c) the printed enclosure zip-tied to the bō.

2. BUILDING THE INSTRUMENT

Inside the enclosure shown in Figure 1, a SparkFun ESP32-S2 Thing Plus microcontroller reads a nine-degrees-of-freedom MPU-9250 inertial sensor over I2C and packs orientation, acceleration, and gyroscope readings into an OSC packet, sent over UDP at 100 Hz across the microcontroller's own connection to the host computer's Wi-Fi hotspot. Joining the host's own hotspot rather than a dedicated access point keeps pairing to a single step, at the cost of a Wi-Fi network name, password, and destination address that are presently compiled into the firmware rather than set at run-time. Nothing tethers the performer to a fixed point on stage; a cable is needed only to charge the unit between sets.

Because the sensor can be glued to the staff at any angle (exact placement is never perfectly reproducible by hand, and often changes from one build of the enclosure to the next), the plugin calibrates itself the first time the staff is held through three reference poses. All three poses together fix which physical axis is functioning as "forward": the plugin scores every candidate axis by how close to mutually orthogonal its direction turns out across the three poses, and keeps whichever axis comes closest. The first two poses then resolve any leftover twist around that axis, by solving for the small rotation that best lines up the measured forward/up directions with the instrument's own ideal frame. From then on, the same instrument logic runs correctly no matter where on the staff the sensor happens to sit, so a performer can move it without



This work is licensed under Creative Commons Attribution – Non Commercial 4.0 International License. The full terms of the License are available at <http://creativecommons.org/licenses/by-nc/4.0/>

re-learning the instrument underneath.

Two recent changes tightened the path from a sensor packet to audible sound. First, the plugin now treats the incoming stream as stale after 150 ms without a fresh packet, rather than the previous 500 ms, so a real Wi-Fi dropout is caught and muted noticeably sooner. Second, that mute no longer shares a single ramp with normal playing: losing the signal now fades the master gain out over 120 ms (long enough to avoid an audible click) while recovering it fades back in over a much shorter 10 ms, so the instrument goes quiet gracefully on dropout but comes back almost immediately once the connection returns.

3. HOW MOTION BECOMES SOUND

Every packet from the sensor is read against a moving reference: not just where the staff is, but how it got there. Four continuous qualities borrowed from Laban Movement Analysis are extracted from the motion stream every audio block. How forcefully the staff moves becomes Weight, read from a leaky integral of its acceleration once gravity is subtracted out, and typically driving loudness or waveshaping. How suddenly a spin speeds up or stops becomes Time, driving attack speed and how quickly a chord change lands. Whether the staff keeps spinning around the same axis or wanders becomes Space, a timbral focus that tightens or loosens a filter or a chorus spread. How bound or free the motion is becomes Flow, crossfading between a controlled and a freer, noisier version of the same sound. Tilt and roll bend pitch and blend voices continuously; a spin's lap count sweeps a filter cutoff along a slow curve; a stab or strike, a sharp reversal in the staff-tip's acceleration, triggers a transient, a chord change, or a percussive burst.

None of the relationships described above is a single fixed algorithm baked into compiled code. Every one of them is authored as a small patch of connected building blocks inside the plugin, on a visual canvas in the lineage of Max [4] and Pure Data [5], closer in feel to the boxes-and-cables idiom that community modular-synthesis software such as VCV Rack [6] carries over from hardware Eurorack patching than to a conventional plugin's fixed control panel. Three functional kinds of node populate that canvas: *sources* (the raw sensor axes and the derived Laban qualities above), *math* (arithmetic; shaping curves such as range-mapping, logarithmic range-mapping, clamping, crossfading, and scale-quantizing; lookup tables; and a handful of stateful primitives - one-pole smoothers, leaky integrators, derivative detectors, hysteresis gates, and a pseudo-random sample-and-hold), and *sinks* (the exposed parameters of whichever synthesis voice the patch is driving). A fourth kind, *display*, carries no signal of its own: a numeric readout, a level meter, and a scrolling scope can be dropped inline on any wire purely to watch its live value, without altering the patch. A patch is a directed graph with no cycles allowed: the plugin topologically sorts it once whenever a wire changes, then re-walks that fixed order once per audio block, so dragging in a node or dropping a new connection mid-performance never forces the audio thread to replan anything. A named preset is nothing more than one such graph, serialised to an XML file and reloaded by name - the same mechanism a performer's own live patch is saved through, so a custom mapping built during a rehearsal is stored no differently from a factory one.

That shared vocabulary is small enough to describe in full, yet stretches from the simplest mapping in the catalog to a considerably more elaborate one. *Simple* is close to the shortest patch the graph can express: the pitch source feeds one range-mapping node that becomes the root frequency (wired straight into the addi-

tive voice's sink), and the roll source feeds an absolute-value node into a second range-mapping node that becomes the gain (wired into its own, separate gain sink) - two motion sources and three math nodes feeding two sink nodes in total, wired once and left alone for the rest of the performance. *Azimuth Kinetic* sits at the other end of the same vocabulary. Its pitch and gain follow the same lookup-table-driven facing logic as the rest of the Azimuth family, but its timbre is deliberately one-to-many [7]: a single smoothed rotation-speed value fans out simultaneously into the filter cutoff, harmonic brightness, waveshaping drive, noise amount, vibrato and tremolo depth, and reverb send, so a spin sounds progressively buzzier, noisier, and more reverberant the faster it turns, rather than moving one parameter in isolation. Layered on top of that energy value, a derivative node watches the continuous spin-lap counter for the instant each full rotation completes and fires a one-block pulse into a sample-and-hold node, which then latches four fresh pseudo-random values; each value is chased by its own slow one-pole "drift channel" - closing roughly 5% of the remaining distance to its new random target on every audio block, so the drift glides rather than jumps - and folded back, in a small proportion, onto the cutoff, drive, noise amount, and harmonic brightness the energy value already set. The result is a mapping that never quite repeats itself: two visually identical spins land on two different, nearby timbres, because every completed rotation quietly re-rolls where the tone drifts to next - an instability the graph's existing math nodes are enough to build, with no bespoke code and no audio-thread cost beyond four extra smoothers.

What a patch's output actually sounds like depends on which of three synthesis voices its sink nodes are wired into. A default additive engine sums four voices of six phase-locked harmonics each, vibrato- and chorus-detuned into a warm, faintly unstable choir that a mapping can drive into a rasping saturation or filter down to a single dark hum; a short comb resonator gives every voice a string-like colouration. That same engine also carries its own built-in noise generator and filter, and it is this path, not a separate voice, that *Wind Noise* actually drives: the patch wires nothing at all into the engine's pitch inputs, only motion-energy-shaped noise and filter cutoff, for a windy, textural character with no fixed pitch centre at all. A second, granular engine sits alongside the additive one, synthesising its own two-second bed of shifting harmonics and filtered noise and scattering up to 64 overlapping grains across it (forward, backward, panned, re-pitched); it is fully implemented but not yet wired into any preset in the current catalog, so it stays silent until a future patch calls on it. A third engine ports Neil Thapen's *Pink Trombone* [8], a digital-waveguide vocal tract excited by a glottal-pulse model and breathed into with filtered noise, so the instrument can also simply speak, or half-speak, rather than only sing - the one engine with a preset built specifically around it, *Vocal Tract*. An engine a patch never wires into is skipped entirely for that block, so presets built on one voice never pay for the other two. Figure 2 shows one such patch in full, exactly as it appears on the plugin's own canvas.

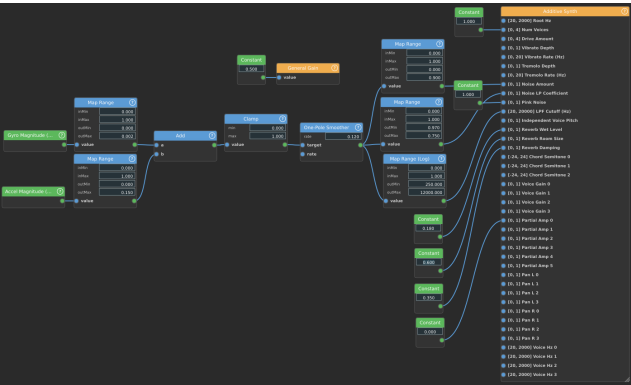


Figure 2: The *Wind Noise* preset, seen inside the plugin: motion sources (green) - gyroscope and accelerometer magnitude - run through shaping and smoothing math (blue) into the exposed parameters of the additive-synthesis voice (orange). There is no pitch input anywhere in this patch: only motion energy, shaping noise amount and the master filter cutoff.

4. A VOCABULARY OF PRESETS

Nine named presets currently ship with the instrument (Table 1), graded by the physical vocabulary they ask of a performer rather than uniform in difficulty. *Simple* asks only for basic tilt and roll, an easy way in that needs no martial background at all. The *Azimuth* and *Martial* families reward the staff-handling skill a martial artist already carries onto a stage: they derive a facing direction and a spin quality straight from the calibrated motion stream, without ever touching a magnetometer (which a stage rig’s own electronics would only confuse), so that where the staff points and how it turns becomes part of the music. *Wind Noise* trades pitch for a breathy, filtered-noise texture sustained by nothing but motion; *Vocal Tract* pushes the same sensor stream into something closer to a voice than a synthesiser. Each preset is less a separate program than a different character the same instrument can take on, chosen for a set the way a performer chooses which piece to play next, rather than a fixed, single-piece score.

This catalog is smaller than it once was. Fifteen mappings were implemented over the course of development, but several turned out to be near-duplicates of one another once built - a plain spin-driven pentatonic melody and its Laban-shifted cousin, three separate Azimuth variants that each added one extra timbral trick (a spin-count filter sweep, a speed-tracking filter, a motion-driven reverb) on top of an otherwise identical mapping. Rather than ship all fifteen, the list was pruned to nine: *Azimuth Kinetic* folds every one of those three Azimuth timbral tricks into a single one-to-many mapping (§3) instead of shipping them as separate, thinner presets, and similar consolidations retired the redundant spin- and momentum-only mappings. The six retired mappings still exist as graphs in the source tree - useful engineering reference points and starting patches to fork from - but are no longer surfaced in the plugin’s own preset selector.

Preset	Character
<i>Simple</i>	Pitch from tilt, volume from roll; one plain sine voice.
<i>Wind Noise</i>	No pitch; motion energy swells filtered noise into gusts.
<i>Bowed Chord</i>	Chromatic root from pitch; yaw/roll pick the chord; spin speed drives it like bow pressure.
<i>Lead + Drone</i>	Major-scale lead over a 4-voice drone; yaw crossfades two drone voices.
<i>Martial Momentum</i>	Root from spin plane/direction; glide speed set by spin momentum.
<i>Azimuth Kinetic</i>	Facing-based root; a single rotation-speed value drives filter, drive, noise, and reverb at once, with per-rotation stochastic drift.
<i>Speed Gate</i>	Slow: pentatonic melody. Fast: crossfades into the Azimuth facing/spin-based root and filter mapping.
<i>Spin Voices (Scale)</i>	Roll picks 1 of 4 scale-quantized voices; the rest hold their last pitch/gain.
<i>Vocal Tract</i>	Physically-modeled voice; pitch/roll/yaw shape glottal pitch and vowel; jabs add consonant-like bursts.

Table 1: The nine presets currently shipping, in load order.

5. WHERE THIS STANDS

Everything above is grounded in the system as it runs today, exercised continuously through the first author’s own development and rehearsal. What it is not yet grounded in is anyone else’s experience of the instrument: no performer besides its designer has sat down with the staff, and the claims above should be read as implementation-level evidence, not validated usability. A semi-structured session with performers who have some staff, baton, or movement background (but no prior exposure to REMORA) is planned next: a calibration walkthrough; free play across a small span of presets chosen to range from *Simple* to a fully Laban-driven one; a short guided task in the node-graph editor itself, retuning an exposed parameter and then rewiring which motion quality drives it, to test whether the patching layer is usable by someone other than its author; and a closing interview. That session is aimed squarely at the question this paper cannot yet answer: whether the instrument is playable, and its motion vocabulary and its patching surface both legible, to someone other than the person who built it.

The instrument also carries limitations worth naming plainly: Wi-Fi pairing is still a manual, reflash-to-change affair rather than a runtime setting, even after the dropout handling itself was tightened (§2); yaw drift is bounded to sessions of tens of minutes by assumption rather than measurement; the single inertial sensor gives REMORA no sense of a second performer, so it remains a solo instrument for now; and the node-graph editor, while it is what makes the preset catalog reproducible and extensible at all, exposes that same complexity to a performer who only wants to pick a preset and play, with no simplified performance-only view that hides the graph until it is asked for.

One extension is next on the list regardless of that session's outcome: replacing the compiled-in Wi-Fi hotspot join with an ESP-NOW peer-to-peer link between the staff's microcontroller and a second one tethered to the host, which would drop the SSID, password, and destination IP address currently baked into firmware in favour of pairing by MAC address, at the cost of one extra piece of hardware in the signal chain.

6. ACKNOWLEDGMENT

This work was carried out during the first author's internship at CREATE, Aalborg University, Denmark. Firmware, plugin source, enclosure files, and the current preset catalog (plus the retired mappings kept as reference graphs, §4) are available at <https://github.com/TimotheeDvt/REMORA>.

7. REFERENCES

- [1] Joseph Malloch and Marcelo M. Wanderley, "The T-Stick: From musical interface to musical instrument," in *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*, New York, USA, 2007, pp. 66–69, DOI: 10.1145/1279740.1279751. [Online]. Available: https://www.nime.org/proceedings/2007/nime2007_066.pdf
- [2] Teresa Marrin and Joseph Paradiso, "The digital baton: A versatile performance instrument," in *Proceedings of the International Computer Music Conference (ICMC)*, Thessaloniki, Greece, 1997. [Online]. Available: <https://quod.lib.umich.edu/i/icmc/bbp2372.1997.083>
- [3] Sangbong Nam, "Musical Poi (mPoi)," in *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*, Daejeon, Korea, 2013, pp. 148–151. [Online]. Available: https://www.nime.org/proceedings/2013/nime2013_254.pdf
- [4] Miller Puckette, "The patcher," in *Proceedings of the International Computer Music Conference (ICMC)*, San Francisco, CA, USA, 1988, pp. 420–429.
- [5] —, "Pure data: Another integrated computer music environment," in *Proceedings of the Second Intercollege Computer Music Concerts*, Tachikawa, Japan, 1996, pp. 37–41.
- [6] Andrew Belt and VCV, "VCV Rack: Open-source virtual modular synthesizer," <https://vcvrack.com>, 2017, software, first released September 2017.
- [7] Andy Hunt and Marcelo M. Wanderley, "Mapping performer parameters to synthesis engines," *Organised Sound*, vol. 7, no. 2, pp. 97–108, 2002, DOI: 10.1017/S1355771802002030.
- [8] Neil Thapen, "Pink Trombone: A voice synthesizer," <https://dood.al/pinktrombone/>, 2017, web-based digital-waveguide vocal tract physical model.